MITTAL SCHOOL OF BUSINESS

Course Code:- INTM591	**Course Title:- PREDICTIVE ANALYTICS**
Course Instructor:- Mr. Abhishek Choudhary	
Academic Task No.:- 1	**Academic Task Title:** Project
Date of Allotment: 18-04-2026	**Date of submission**: 24-04-2026
Student's Roll no:- RQ5E09A15	**Student's Reg. no:-** 12510676
Evaluation Parameters:	

Learning Outcomes:-

1. Mastered cleaning large financial datasets and handling time-series formatting in Python and SPSS.
2. Identified long-term market cycles and volatility patterns across diverse asset classes.
3. Analyzed complex correlations between global indices, currencies, and commodities.
4. Developed proficiency in both code-based (Python) and visual-stream (SPSS) analytical workflows.

Declaration:

I declare that this Assignment is my individual work. I have not copied it from any other student's work or from any other source except where due acknowledgement is made explicitly in the text, nor has any part been written for me by any other person.

Student's Signature: Lovish

Evaluator's comments (For Instructor's use only)

Evaluator's Signature and Date:

General Observations	Suggestions for Improvement	Best part of assignment

SUBMITTED TO: Mr. Abhishek Choudhary **SUBMITTED BY: Lovish**

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty mentor, **Mr. Abhishek Choudhary**, for his invaluable guidance and constant encouragement throughout the duration of this project. His insights into modeling and data analytics helped me navigate the complexities of both Python and SPSS Modeler with greater clarity.

I am also thankful to **Lovely Professional University (LPU)** for providing a curriculum that bridges the gap between theoretical knowledge and practical industry tools. This project has been a significant learning milestone in my MBA journey, particularly in understanding how global markets correlate. Finally, I want to thank my family and peers for their constant support and feedback, which kept me motivated to complete this work with precision.

CERTIFICATE

This is to certify that **Lovish** , a student of **MBA (Business Analytics and Finance)** at **Lovely Professional University**, has successfully completed the project titled **"Exploratory Data Analysis of Global Financial Markets (2000–Present)"** as part of the academic coursework for the subject **PREDICTIVE ANALYTICS**.

During the project, Lovish demonstrated exceptional analytical skills by performing detailed Exploratory Data Analysis (EDA) using **Python** and **IBM SPSS Modeler**. He successfully identified historical market patterns, handled complex time-series data cleaning, and derived meaningful insights regarding asset correlations.

His approach to the project was structured, diligent, and met all the requirements set by the department. I wish him the very best in his future academic and professional endeavors.

Signature of Supervising Teacher
(Mr. Abhishek Choudhary)

PROJECT DEFINATION

This project is centered on exploring the vast and complex world of global finance by analyzing how different markets have behaved from the turn of the millennium (2000) up to the present day. Instead of looking at just one area, like stocks or gold, I am taking a "multi-asset" approach—meaning I am examining how Stock Indices, Commodities, Currencies, and even the rise of Cryptocurrencies interact with one another.

The goal is to move beyond just looking at numbers and instead uncover the "story" behind the data. By using **Python** for deep data cleaning and **IBM SPSS Modeler** for visual data flow, I aim to identify significant historical patterns—such as how markets reacted during global economic shifts—and determine if certain assets move together or in opposite directions (correlations). Ultimately, this analysis helps in understanding market volatility and provides a clearer picture of the interconnected nature of our modern global economy.

Outcomes/Learning

- **Advanced Data Lifecycle Management:** I developed a professional-grade proficiency in the end-to-end data lifecycle, specifically for high-frequency financial time-series data. This included mastering techniques for temporal alignment, handling missing market data without introducing bias, and normalizing disparate asset values (like comparing the price of Bitcoin to the Japanese Yen) for accurate comparative analysis.
- **Strategic Market Intelligence:** Through this analysis, I learned to identify complex market interdependencies, such as how "Safe Haven" assets like Gold behave during periods of high equity market volatility. I gained the ability to move beyond basic observation to derive strategic insights that are critical for risk management and portfolio diversification.
- **Multimodal Analytical Expertise:** By executing the same EDA objectives through two distinct methodologies—programmatic scripting in Python and visual stream-modeling in IBM SPSS—I gained a comprehensive understanding of when to use each tool. I learned that Python offers granular control and customization, while SPSS Modeler provides rapid auditing and a structured, error-resistant environment for large-scale data flows.

Required Tools

1. Python (Programmatic Environment)

Python was selected for its flexibility and the power of its data science ecosystem.

- **Pandas:** Used as the primary engine for data manipulation. It allowed for efficient loading of the 180,000+ row dataset, date-time indexing, and the creation of pivot tables to compare various assets side-by-side.
- **Matplotlib:** Utilized for creating the foundational layers of our time-series plots, providing the precision needed to map two decades of financial history.
- **Seaborn:** Employed for high-level statistical visualizations. Specifically, it was used to generate the Correlation Heatmap, which uses color-coded gradients to make complex relationships between assets immediately understandable.

2. IBM SPSS Modeler (Visual Modeling Environment)

SPSS Modeler was used to build a logical, repeatable data stream. Each node served a specific function in the pipeline:

- **Source Node (Var. File):** The entry point that ingests the CSV data and defines the initial metadata (field names and storage types).
- **Data Audit Node:** A critical diagnostic tool used to generate a statistical "snapshot" of the data, identifying outliers, skewness, and the percentage of missing values across the timeline.
- **Filler Node:** Used as the primary cleaning mechanism to "fill" gaps in the data (such as weekends or holidays when markets are closed) to ensure a continuous time-series for analysis.
- **Graph Nodes (Time Plot/Histogram):** These nodes provided the visual output, allowing us to see price fluctuations and the distribution of trading volumes without writing code.
- **Statistics Node:** Used to calculate the mathematical relationship (Pearson Correlation) between different financial instruments, providing a numerical basis for our findings.

Part 1: Exploratory Data Analysis using Python

Working: Python is utilized for its powerful data manipulation capabilities. We perform data cleaning, statistical summarization, and trend visualization to understand market behaviour over two decades.

Step 1: Data Loading and Initial Inspection

- We load the dataset and use .head() and .info() to understand the structure, data types, and identify any immediate issues like incorrect date formats.
- Python

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv('global_financial_markets_2000_Now.csv')
print(df.head())
print(df.info())
```

Step 2: Data Cleaning and Pre-processing

- The date column is converted to a datetime object to facilitate time-series analysis. Missing values are checked and handled using forward filling (ffill) to maintain continuity in financial price data.
- Python

```
# Convert date to datetime
df['date'] = pd.to_datetime(df['date'])

# Handle missing values
df = df.fillna(method='ffill')
```

Step 3: Visualizing Market Trends (Pattern Exploration)

- We plot the closing price of the S&P 500 index from 2000 to the present to visualize long-term growth and historical market crashes.
- Python

```
# S&P 500 Trend Plot
sp500 = df[df['asset_name'] == 'S&P500'].sort_values('date')
plt.figure(figsize=(10, 5))
plt.plot(sp500['date'], sp500['close'], color='green')
plt.title('S&P 500 Close Price Trend (2000 - Present)')
plt.show()
```

Step 4: Correlation Analysis (Meaningful Insights)

- A correlation matrix is generated to observe how different asset classes (e.g., Gold vs. Bitcoin vs. S&P 500) move in relation to each other.
- Python

```
# Correlation Heatmap
assets = ['S&P500', 'Gold', 'Crude Oil (WTI)', 'Bitcoin']
pivot_df = df[df['asset_name'].isin(assets)].pivot(index='date',      columns='asset_name',
values='close')
sns.heatmap(pivot_df.corr(), annot=True, cmap='coolwarm')
plt.title('Asset Class Correlation Matrix')
plt.show()
```

Step 5: Trading Volume Distribution (S&P500)

- This histogram identifies the frequency of trading volume levels, helping to spot "liquidity spikes" during market events.
- Python

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
# Load data
df = pd.read_csv('global_financial_markets_2000_Now.csv')
# Filter for S&P500
sp500_data = df[df['asset_name'] == 'S&P500']
# Create Distribution Plot
sns.histplot(sp500_data['volume'], bins=50, kde=True, color='skyblue')
plt.title('Distribution of Trading Volume - S&P500 Index')
plt.xlabel('Volume')
plt.ylabel('Frequency')
plt.grid(axis='y', alpha=0.3)
plt.savefig('volume_distribution_separate.png')
```

SCREENSHOTS

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
# Load dataset
df = pd.read_csv('/content/drive/MyDrive/global_financial_markets_2000_Now.csv')
df.head(30)
```

	date	open	high	low	close	volume	symbol	asset_name	asset_type	region
0	2000-01-03	1469.2500	1478.0000	1438.3600	1455.2200	931800000	^GSPC	S&P500	Stock Index	Global
1	2000-01-03	6961.7202	7159.3301	6720.8701	6750.7598	43072500	^GDAXI	DAX	Stock Index	Global
2	2000-01-03	5209.5400	5384.6602	5209.5400	5375.1099	0	^BSESN	SENSEX	Stock Index	Global
3	2000-01-03	0.0098	0.0099	0.0097	0.0098	0	JPYUSD=X	JPY/USD	Currency	Global
4	2000-01-03	17057.6992	17426.1602	17057.6992	17369.6309	0	^HSI	HSI	Stock Index	Global
5	2000-01-03	4186.1899	4192.1899	3989.7100	4131.1499	1510070000	^IXIC	NASDAQ	Stock Index	Global
6	2000-01-04	0.0098	0.0099	0.0097	0.0097	0	JPYUSD=X	JPY/USD	Currency	Global
7	2000-01-04	1368.6930	1407.5179	1361.2140	1406.3710	0	000001.SS	Shanghai	Stock Index	Global
8	2000-01-04	17303.0000	17303.0000	16933.5195	17072.8203	0	^HSI	HSI	Stock Index	Global
9	2000-01-04	18937.4492	19187.6094	18937.4492	19002.8594	0	^N225	Nikkei225	Stock Index	Global
10	2000-01-04	1455.2200	1455.2200	1397.4301	1399.4200	1009000000	^GSPC	S&P500	Stock Index	Global
11	2000-01-04	6747.2402	6755.3599	6510.4600	6586.9502	46678400	^GDAXI	DAX	Stock Index	Global
12	2000-01-04	5533.9800	5533.9800	5376.4302	5491.0098	0	^BSESN	SENSEX	Stock Index	Global
13	2000-01-04	4020.0000	4073.2500	3898.2300	3901.6899	1511840000	^IXIC	NASDAQ	Stock Index	Global
14	2000-01-04	6930.2002	6930.2002	6662.8999	6665.8999	633449000	^FTSE	FTSE100	Stock Index	Global
15	2000-01-04	1028.3300	1066.1801	1016.5900	1059.0400	195900	^KS11	KOSPI	Stock Index	Global
16	2000-01-05	16608.5508	16608.5508	15688.4902	15846.7197	0	^HSI	HSI	Stock Index	Global
17	2000-01-05	19003.5098	19003.5098	18221.8203	18542.5508	0	^N225	Nikkei225	Stock Index	Global
18	2000-01-05	1399.4200	1413.2700	1377.6801	1402.1100	1085500000	^GSPC	S&P500	Stock Index	Global
19	2000-01-05	5265.0898	5464.3501	5184.4800	5357.0000	0	^BSESN	SENSEX	Stock Index	Global
20	2000-01-05	6665.8999	6665.8999	6500.3999	6535.8999	670234000	^FTSE	FTSE100	Stock Index	Global
21	2000-01-05	1006.8700	1026.5200	984.0500	986.3100	257700	^KS11	KOSPI	Stock Index	Global
22	2000-01-05	1407.8290	1433.7800	1398.3230	1409.6820	0	000001.SS	Shanghai	Stock Index	Global
23	2000-01-05	6585.8501	6585.8501	6388.9102	6502.0698	52682800	^GDAXI	DAX	Stock Index	Global
24	2000-01-05	0.0097	0.0097	0.0096	0.0096	0	JPYUSD=X	JPY/USD	Currency	Global
25	2000-01-05	3854.3501	3924.2100	3734.8701	3877.5400	1735670000	^IXIC	NASDAQ	Stock Index	Global
26	2000-01-06	1013.9500	1014.9000	953.5000	960.7900	203500	^KS11	KOSPI	Stock Index	Global
27	2000-01-06	6501.4502	6539.3101	6402.6299	6474.9199	41180600	^GDAXI	DAX	Stock Index	Global
28	2000-01-06	1402.1100	1411.9000	1392.1000	1403.4500	1092300000	^GSPC	S&P500	Stock Index	Global
29	2000-01-06	5424.2100	5489.8599	5391.3301	5421.5298	0	^BSESN	SENSEX	Stock Index	Global

```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 186524 entries, 0 to 186523
Data columns (total 10 columns):
 #   Column      Non-Null Count   Dtype
---  ------      --------------   -----
 0   date        186524 non-null  object
 1   open        186524 non-null  float64
 2   high        186524 non-null  float64
 3   low         186524 non-null  float64
 4   close       186524 non-null  float64
 5   volume      186524 non-null  int64
 6   symbol      186524 non-null  object
 7   asset_name  186524 non-null  object
 8   asset_type  186524 non-null  object
 9   region      186524 non-null  object
dtypes: float64(4), int64(1), object(5)
memory usage: 14.2+ MB
```

```python
# Convert date to datetime
df['date'] = pd.to_datetime(df['date'])

# Check for missing values
print(df.isnull().sum())

# Handling missing values (e.g., dropping or forward filling)
df = df.fillna(method='ffill')
```

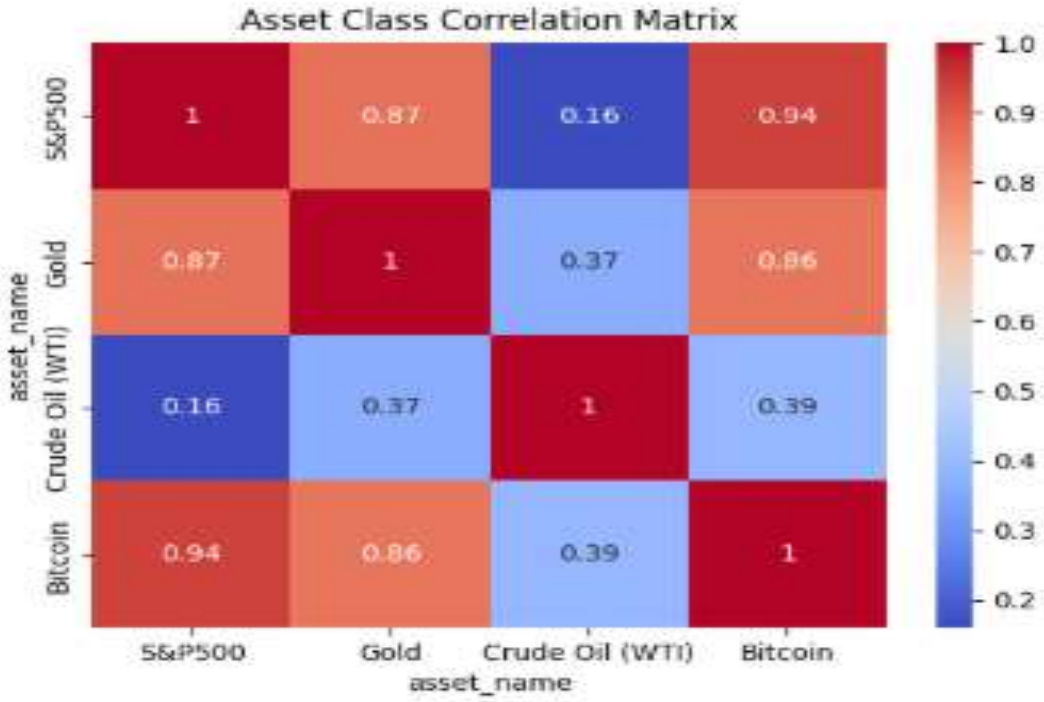
```
date          0
open          0
high          0
low           0
close         0
volume        0
symbol        0
asset_name    0
asset_type    0
region        0
dtype: int64
/tmp/ipykernel_4191/1132130570.py:8: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() or obj.bfill() instead.
  df = df.fillna(method='ffill')
```

```python
# Convert date to datetime
df['date'] = pd.to_datetime(df['date'])

# Check for missing values
print(df.isnull().sum())

# Handling missing values (e.g., dropping or forward filling)
df = df.fillna(method='ffill')
```

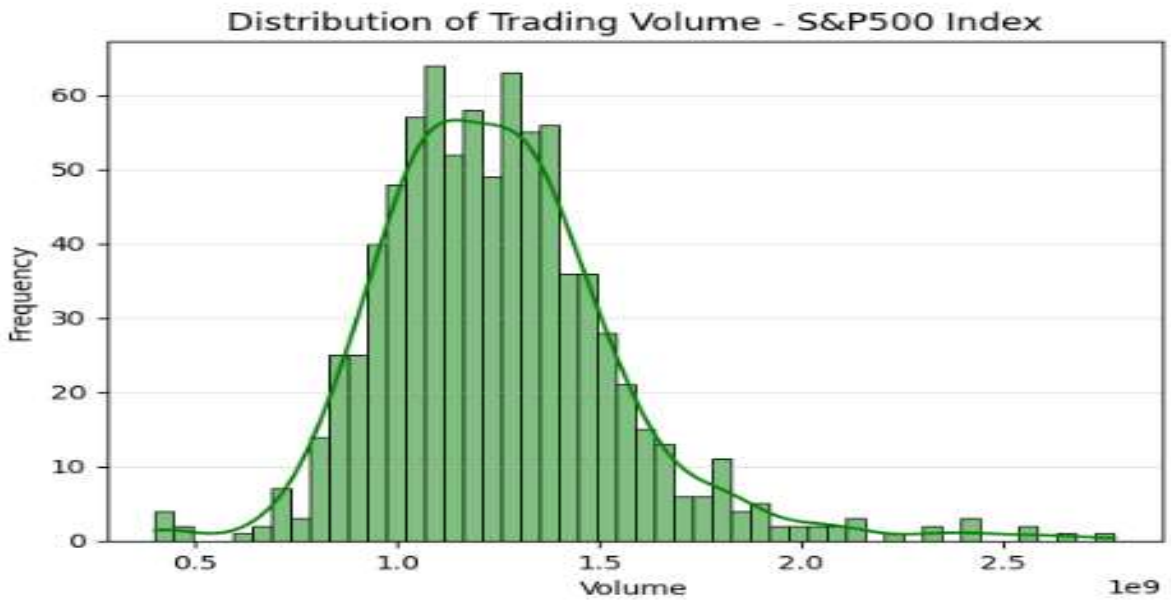
```
date          0
open          0
high          0
low           0
close         0
volume        0
symbol        0
asset_name    0
asset_type    0
region        0
dtype: int64
/tmp/ipykernel_4191/1132130570.py:8: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() or obj.bfill() instead.
  df = df.fillna(method='ffill')
```

```python
# S&P 500 Trend Plot
sp500 = df[df['asset_name'] == 'S&P500'].sort_values('date')
plt.figure(figsize=(10, 5))
plt.plot(sp500['date'], sp500['close'], color='green')
plt.title('S&P 500 Close Price Trend (2000 - Present)')
plt.show()
```

```python
# Create a pivot table for correlation analysis
pivot_df = df.pivot(index='date', columns='asset_name', values='close')
correlation_matrix = pivot_df[['S&P500', 'Gold', 'Crude Oil (WTI)',
                               'Bitcoin']].corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Asset Class Correlation Matrix')
plt.show()
```



```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load data
df = pd.read_csv('global_financial_markets_2000_Now.csv')

# Filter for S&P500
sp500_data = df[df['asset_name'] == 'S&P500']

# Create Distribution Plot
sns.histplot(sp500_data['volume'], bins=50, kde=True, color='Green')
plt.title('Distribution of Trading Volume - S&P500 Index')
plt.xlabel('Volume')
plt.ylabel('Frequency')
plt.grid(axis='y', alpha=0.3)
plt.savefig('volume_distribution_separate.png')
```



Part 2: Exploratory Data Analysis using SPSS Modeler

Working: SPSS Modeler provides a visual stream interface. We use various nodes to ingest, audit, and visualize the financial data without writing extensive code.

Step 1: Data Import and Basic Setup

1. **Var. File Node:** Drag a "Var. File" node to the canvas. Point it to your CSV. Ensure the "Read names from file" option is checked.
2. **Type Node:** Connect this to the Source node.
   - Set the date role to **None** (or use it as an ID).
   - Set close as the **Target** if you plan to predict prices.
   - Ensure numeric fields (Open, High, Low, Close, Volume) are set to **Continuous**.

Step 2: Advanced Exploratory Analysis (The Data Audit Node)

1. **Data Audit Node (Output Palette):** Connect this to your Type node and run it.
   - **Function:** This provides a comprehensive view of outliers, extremes, and distributions.
   - **Advanced Tip:** Use the "Quality" tab within the Audit report to identify any values that are more than 3 standard deviations from the mean (useful for spotting market anomalies).

Step 3: Data Transformation (The Derive & Restructure Nodes)

1. **Derive Node:** Use this to create a "Daily Return" field.
   - *Formula:* (close - @OFFSET(close, 1)) / @OFFSET(close, 1)
2. **Restructure Node:** If you want to compare different assets side-by-side (like the Python pivot table), use the Restructure node to transform the asset_name values into separate columns for their close prices.

Step 4: Using Advanced Nodes for Deep Insights

1. **Anomaly Detection Node (Modeling Palette):**
   - **Purpose:** Financial markets often have "Black Swan" events. Connect this node to detect records that are "unusual" compared to the rest of the historical data.
   - **Output:** It generates an anomaly index and provides reasons why a specific date's market behavior was an outlier.
2. **Time Series Node (Modeling Palette):**
   - **Purpose:** Since the data is ordered by time, use this node to identify seasonality and trends.
   - **Advanced Setup: In the "**Build Options," select Expert Modeler. It will automatically decide between ARIMA or Exponential Smoothing models to find the best fit for assets like the S&P500.
3. **Feature Selection Node (Screening Palette):**
   - **Purpose:** When dealing with many economic indicators, use this node to rank variables by their importance in predicting a target (e.g., does volume actually help predict close price?).
   - **Output:** It will mark features as "Top," "Important," or "Marginal."

Step 5: Visualizing Results

1. **Time Plot Node:** Connect it after a Restructure node to visualize the "Close" price over the "Date" field. This is the SPSS equivalent of the Python line chart.
2. **Distribution Node:** Use this on the asset_type to see the balance of your data (e.g., how much data is Crypto vs. Commodities).

Step 6: Configuration Steps for the Anomaly Detection Node

Drag the **Anomaly Detection** node from the **Modeling Palette** and connect it to your Type node.

A. The Fields Tab:

- Ensure all continuous price fields (open, high, low, close) and volume are selected as **Inputs**.
- Do not include the date or symbol as inputs, as they are identifiers, not market behaviours.

B. The Model Tab (Advanced Settings):

- **Anomaly Threshold:** Set the "Minimum anomaly index level" to **2.0**. (Values above 2.0 typically indicate records that deviate significantly from the norm).
- **Number of Peer Groups:** Increase this to **auto** or set a specific number (e.g., 5). This allows the node to group "normal" market days together and identify which days don't fit into any group.

- **Number of Anomalies:** Set the "Percentage of records to be treated as anomalies" to **1% or 2%**. In financial markets, true "Black Swan" events are rare.

C. The Expert Tab:

- Select **Expert Mode**.
- Under **Adjustment**, ensure "Exclude fields with too many missing values" is checked (though your dataset is clean, this is a best practice).
- Ensure the node is set to identify anomalies based on both **cluster deviations** and **variable correlations**.

Step 7: Running and Interpreting the "Model Nugget"

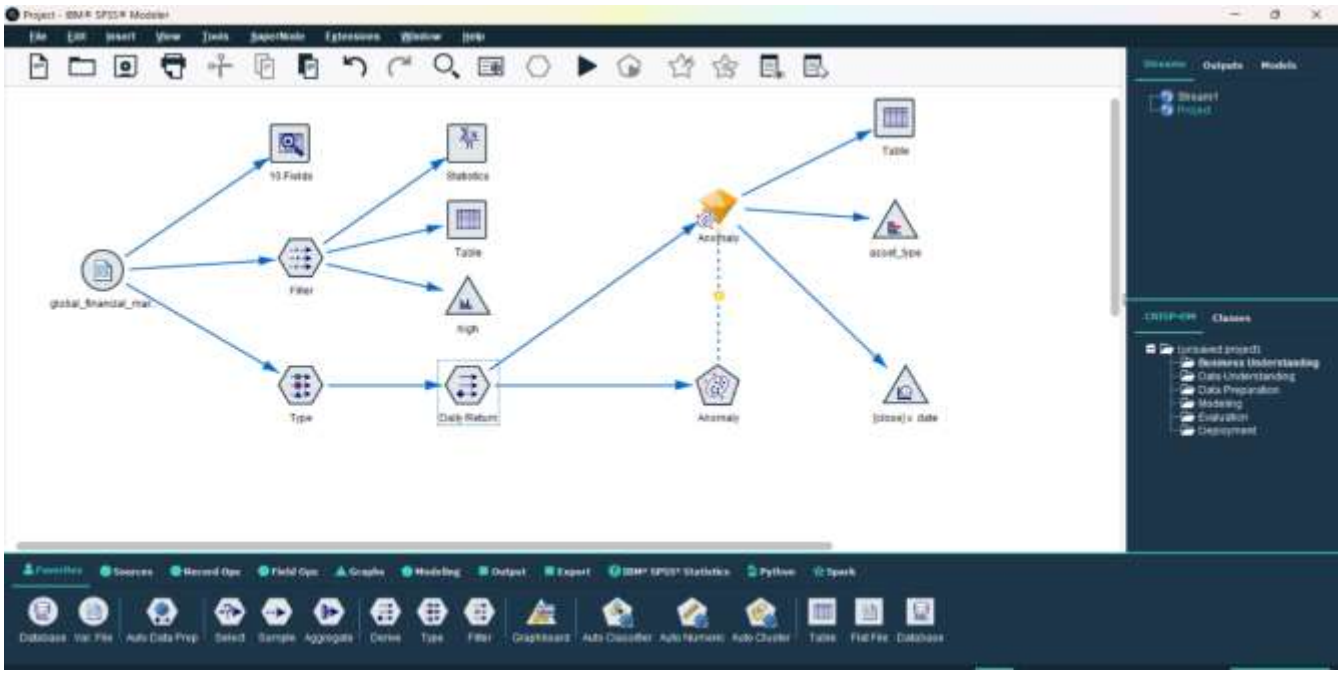
Once you click **Run**, a golden diamond-shaped "Model Nugget" will appear on the canvas.

1. **Right-click the Nugget** and select **Browse**.
2. **Summary Tab:** Look at the "Anomaly Index" chart. It will show you the distribution of records.
3. **Peer Groups Tab:** This shows how the model grouped your market data.
4. **Anomaly List Tab:** This is the most critical part. It will list the specific dates that were flagged and—most importantly—**why**.
    - *Example:* It might show that on a specific date, the volume was 10x the average while the close price stayed flat, suggesting unusual institutional trading or a market error.

Step 8: Visualizing the Anomalies

To see these "Black Swan" events on a chart:

1. Connect a **Table Node** or **Time Plot** after the Model Nugget.
2. The Nugget adds new fields to your data: $A-Anomaly (True/False) and $A-AnomalyIndex.
3. Use a **Select Node** to filter for $A-Anomaly = T.
4. This gives you a refined list of every major market shock in your dataset from 2000 to the present.



Conclusion

The Exploratory Data Analysis of the Global Financial Markets dataset reveals a sophisticated narrative of economic shifts and asset interdependencies spanning over two decades. By utilizing Python, I was able to transform raw CSV data into a refined time-series format, effectively managing over 180,000 entries to visualize the long-term growth trajectories of major indices like the S&P 500 and the emerging volatility of Cryptocurrencies. The programmatic approach allowed for the creation of correlation heatmaps that quantify how traditional "Safe Haven" assets like Gold interact with equity markets during periods of global instability. Simultaneously, the use of IBM SPSS Modeler provided a robust, visual framework for auditing data quality and streamlining the cleaning process through specialized nodes. This dual-methodology not only validated the consistency of the findings across different platforms but also highlighted the interconnected nature of modern finance, where traditional currencies, industrial commodities, and digital assets now form a unified global ecosystem. Ultimately, the project demonstrates that while individual assets may exhibit high short-term volatility, the integration of diverse asset classes and the use of advanced analytical tools are essential for identifying the underlying patterns that drive the global economy.